

Week 4: Microcontroller Programming

Arduino Fundamentals

Programming Basics

Languages are made up of several fundamental elements like verbs, nouns, adjectives, etc.

Programming languages are the same way, except with:

- Data types
- Variables
- Basic operations
- Conditional statements
- Loops
- Functions

Data Types

Different types of data:

Data Type	Example
int	333
float	0.003
long	3333333333333333
char	'k'
String	"Hello, World!"
bool	true, false

Variables

Just like in math, variables store values.

- Variables must be **declared** before they are used
- Variables can be **assigned** a value many times, but only need to be declared once
- Variables should have names that describe their content
- A data type is declared before the variable name
- Pin numbers are `ints` even if they're D0

Declaration

```
int number = 2;  
int anotherNumber = 4;  
String text = "Hello";
```

Assignment

```
number = 5;  
anotherNumber = number;  
text = "Hi";
```

Variables???

How would you...

- Declare a variable `num` to store the value `100` (or what data type?)
- Declare a variable `isOn` to store the value `true`
- Declare a variable `multiplier` to store the value `0.01`
- Assign the variable `greeting` the value `"hi"`
- Declare the variable `ledPin` to store the value `D0`

Basic Operations

Operations tell the microcontroller to perform some mathematical, relational, or logical operation.

```
int a = 5;  
int b = 6;  
a = b - a;
```

Result: a is now equal to 1

Arithmetic operators

Operator	Meaning
-	Subtraction
*	Multiplication
/	Division
%	Modulus (Remainder of division)
++	Increment by 1 (+1)
--	Decrement by 1 (-1)
+=	Increment by some #
-=	Decrement by some #

Basic Operations

```
int a = 5;  
int b = 6;  
bool x = a < b;
```

Result: x is now equal to true

Relational Operators

Operator	Meaning
==	Is equals to
!=	Is not equals to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Basic Operations

```
bool t = true;  
bool f = false;  
bool x = t && f;
```

Result: x is now equal to false

Logical Operators

Operator	Meaning
&&	AND
	OR
!	NOT

Basic Operations???

- How to assign the value of x to be the sum of variables a and b?
- What is the value of (!true || false)?
- What is value of 100 % 10?
- What is the value of the following code snippet:

```
int x = 5;
```

```
x++;
```

```
x++;
```

```
x += 5;
```

```
x--;
```

Conditional statements

- Executes code based on whether a condition is met or not
- You can have conditional statements inside conditional statements

Example

```
if (button == true) {  
    // Turn LED on  
} else {  
    // Turn LED off  
}
```

Code inside curly brackets gets executed if the boolean button has the value true

Conditional statements

- You can have multiple `if` checking statements using `else if`

Example

```
if (button == true) {  
    // Make LED red  
} else if (button2 = true) {  
    // Turn LED blue  
} else {  
    // Turn LED off  
}
```

Only one code segment will be executed. Order matters!

Conditional statements

I want to check when a given variable `x` is odd or even. I want to store “odd” in `solution` if `x` is odd and “even” in `solution` if `x` is even.

```
if (???) {  
    solution = "even";  
} ??? {  
    solution = "odd";  
}
```

Loops

Loops are useful for executing lines of code multiple times. Suppose I want to double a variable `x` **five** times.

We could hard code it:

```
x = x * 2;  
x = x * 2;  
x = x * 2;  
...
```

...but it's really tedious to do it manually and it doesn't look clean.

Loops

Instead we could use a `while()` loop. As long as the condition inside is satisfied, the code inside the curly brackets will be executed.

Example

```
int i = 0;
while (i < 5) {
    x = x * 2;
    i++
}
```

We double x when i = 0

We double x when i = 1

We double x when i = 2

We double x when i = 3

We double x when i = 4

At i = 5, we stop doubling x.

Loops

We could clean this up even more with a for loop.

Example

Iterative variable

Condition

Executed at the
end of code in
curly brackets

```
for (int i = 0; i < 5; i++) {  
    x = x * 2;  
}
```

Loops???

What would we change if we wanted the code to execute 10 times?

```
for (int i = 0; i < 5; i++) {  
    x = x * 2;  
}
```


Functions

A function is a block of reusable code. We specify the **return type**, **name of the function**, and **function parameters**. **Order matters!!!**

Example

```
float slopeCalc(int x1, int x2, int y1, int y2) {  
    int top = y2 - y1;  
    int bottom = x2 - x1;  
    float slope = top / bottom;  
    return slope;  
}
```

Functions

Now we can call the function using the **name of the function** and supplying necessary **function parameters**. We also expect the **return type** in the output. **Again, order matters!!!**

Example

```
int point1x = 5  
int point1y = 10  
int point2x = 15  
int point2y = 20
```

```
float m1 = slopeCalc(point1x, point2x, point1y, point2y);
```

```
float m2 = slopeCalc(1, 2, 3, 4);
```

Functions???

Given the following function header, give an example function call:

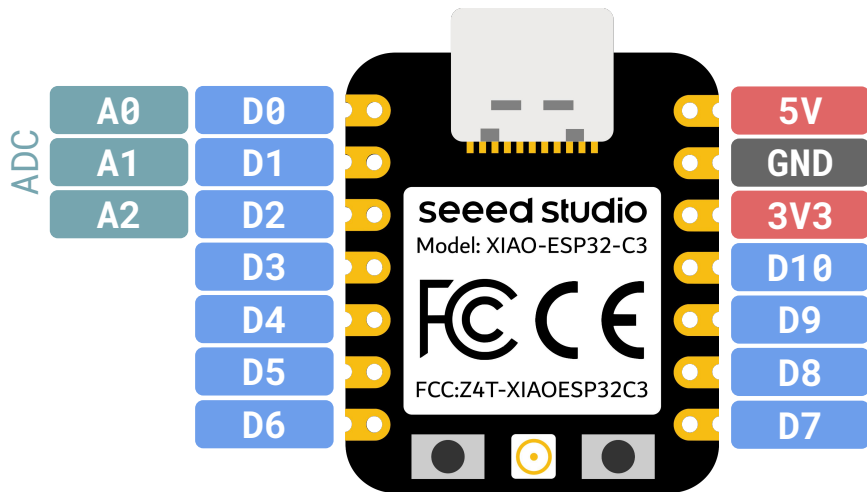
```
float f;  
int i;  
String s;  
Bool b;
```

```
bool mysteryFunc(String a, float b, int c) { ... }
```

What is Arduino and ESP32?

- Arduino and ESP32 are platforms of hardware and software.
- We use ESP32C3 Xiao microcontrollers: mini computers that take inputs, perform outputs, and store a bit of data.
- We program our Xiao microcontrollers using the Arduino “language”. The Arduino language is a modified version of C/C++.

Your ESP32C3 Xiao



On the right hand side you have **5V**, **GND**, and **3V3**.

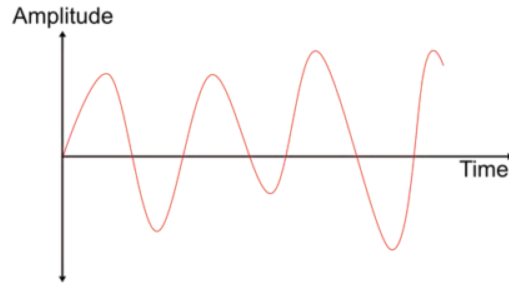
Get in the habit of color coding voltage wires as **red** and GND wires as **black**.

Some components use 5V and some use 3V3. Note that supplying 5V to the GPIO pins can damage your Xiao.

Analog vs. Digital Signals

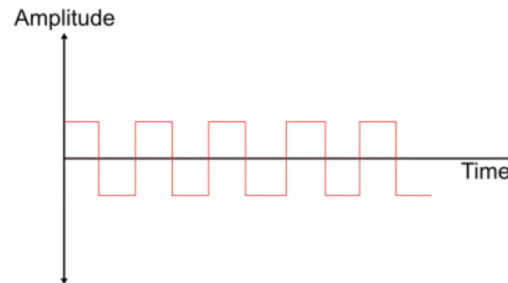
Analog: data that continuously and infinitely varies over time.

Ex: radio waves or sound waves.

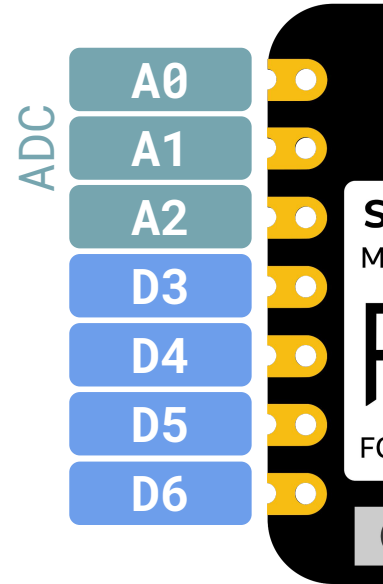


Digital: data that has discrete values. Can only take one value from a finite set of possible values at a given time.

Ex: Binary signal



Only **A0, A1, A2** can read analog signals.



Arduino Functions

The main additions of the Arduino language are functions that allow us to interact our microcontrollers.

Function	What does it do?
<code>digitalRead(pin)</code>	Reads a value from an input pin, either HIGH or LOW
<code>digitalWrite(pin)</code>	Writes a value to a output pin, either HIGH or LOW
<code>analogRead(pin)</code>	Reads a value from an analog input pin. Analog signal
<code>analogWrite(pin)</code>	Writes a value to an output pin, either analog or . Digital signal simulating analog signal via PWM.
<code>pinMode(pin, mode)</code>	Sets pin to mode (ie. set pin D0 to INPUT).
<code>Serial.println()</code>	Prints text to the serial monitor

Microcontroller Signals

HIGH essentially means the pin is connected to some voltage

LOW essentially means the pin is connected to zero voltage (ground)

Analog signals are a range of voltage

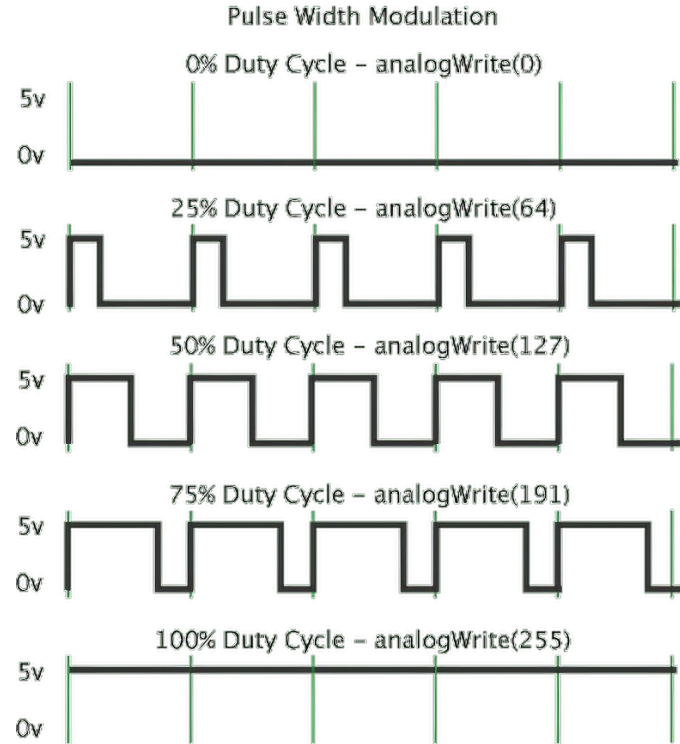
If we wanted to blink an LED (turn it on and off), we could use a `digitalWrite()`.

If we wanted to read the value of a potentiometer, which has varying voltage from 0 to 3V3/5V, we could use an `analogRead()`.

Pulse Width Modulation (PWM)

Your Xiaos can't actually output true analog signals. Instead, they use **pulse width modulation** to get analog results from a digital signal.

Suppose you want to dim an LED but you can only turn the light on or off. One way you can do this is turning the light on and off extremely quickly.



Resetting and Enter Bootloader Mode

Pressing the **Reset** button restarts the code from the top.

If code isn't uploading, you can try entering into bootloader mode:

1. Hold the **Boot** button
2. Press and release the **Reset** button
3. Release the **Boot** button

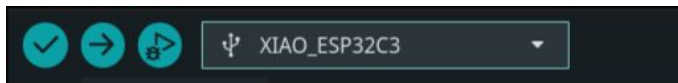
The only way to stop code from running is to unplug the microcontroller or clever programming.



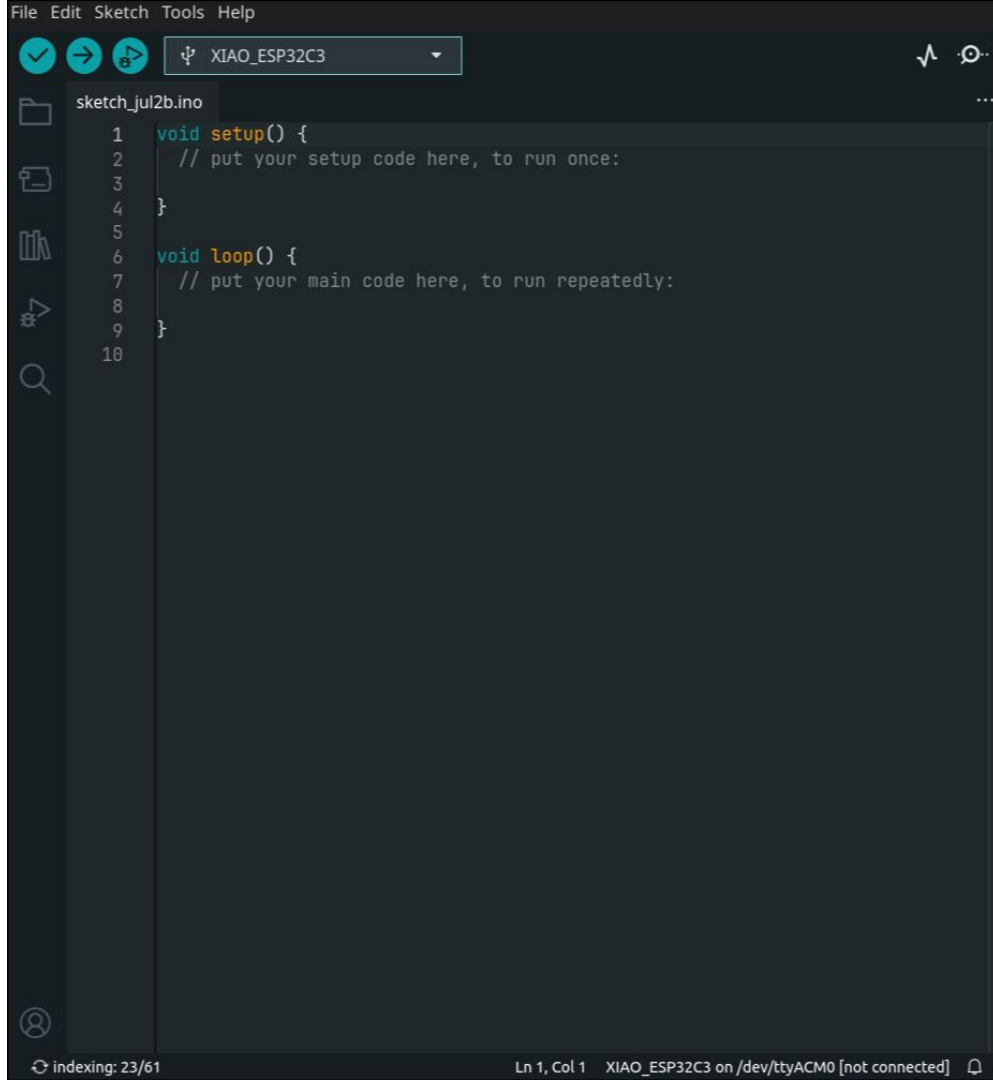
Arduino IDE

Environment where you write, upload, and monitor code.

Select your board, upload code



View the serial monitor



Initialize libraries and variables.



Code in the **setup()** function only gets run once. Start the serial monitor, declare pins, etc.



Code in the **loop()** function runs over and over. Read from input, output actions, etc.



Libraries are code that are written by other people.

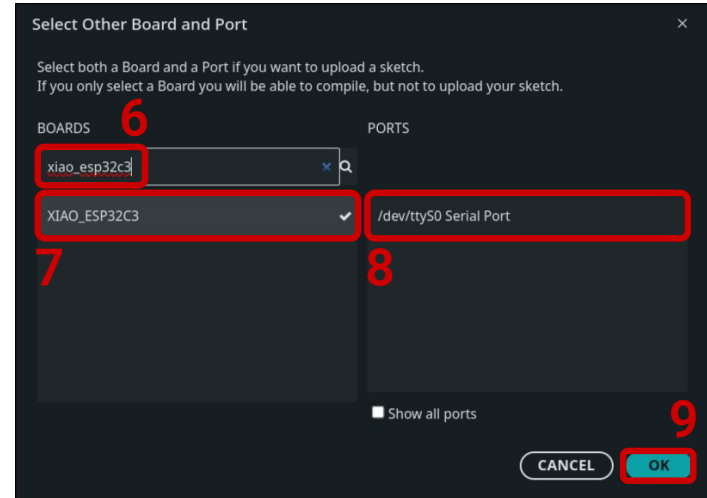
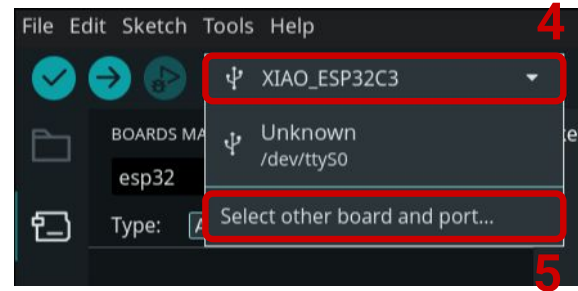
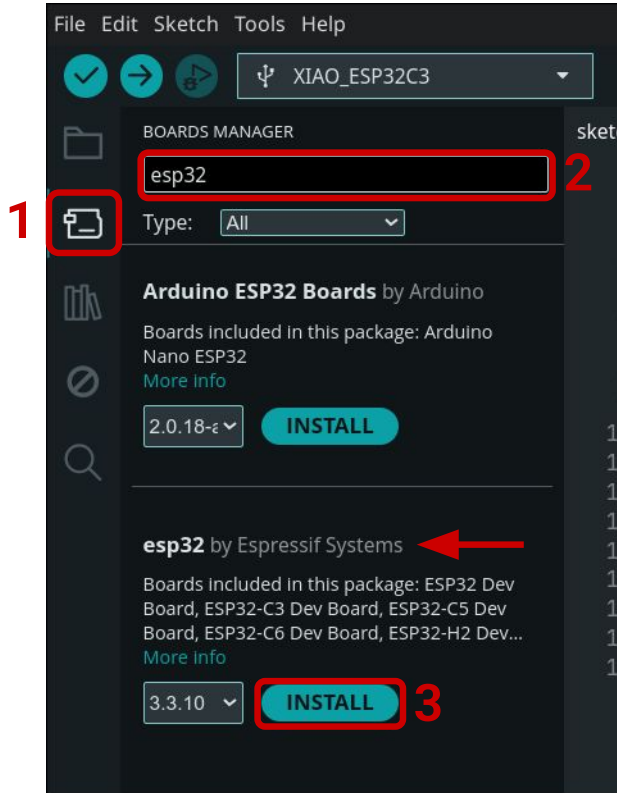
Suppose you want to read data from an SD card. You don't have to handle the low level implementation, you can use an existing library that allows you to read from an SD card with a single function call.

sketch_jul2b.ino

```
1  #include library
2
3  int value;
4  int buttonPin D7;
5  String message = "Hello World";
6
7  void setup() {
8      // put your setup code here, to run once:
9      Serial.begin(9600);
10     pinMode(buttonPin, INPUT);
11     Serial.println(message);
12 }
13
14 void loop() {
15     // put your main code here, to run repeatedly:
16     value = digitalRead(buttonPin);
17     Serial.println(value);
18 }
```

Typical code structure

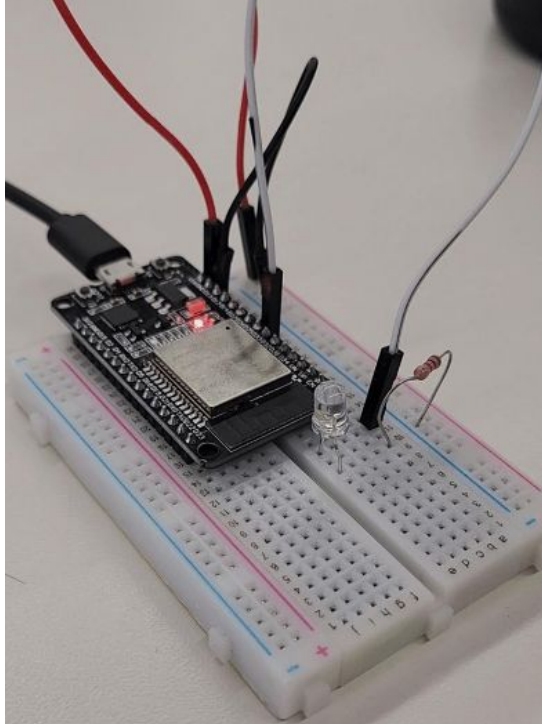
Selecting Your Board



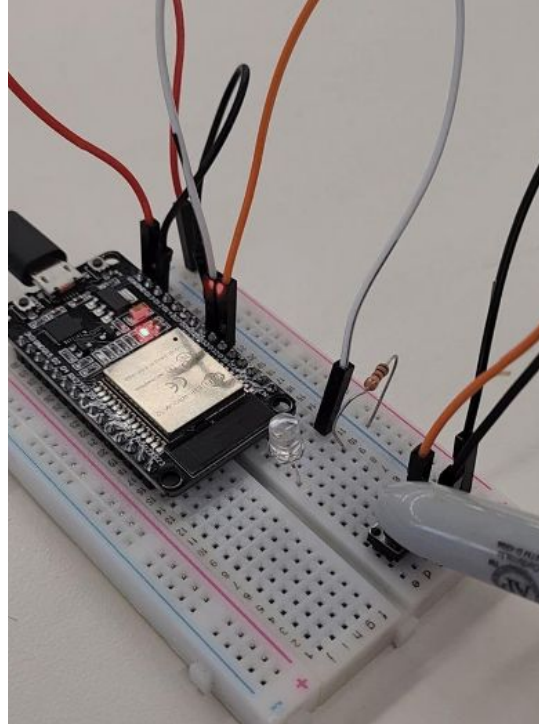
Let's Code!

Coding Examples

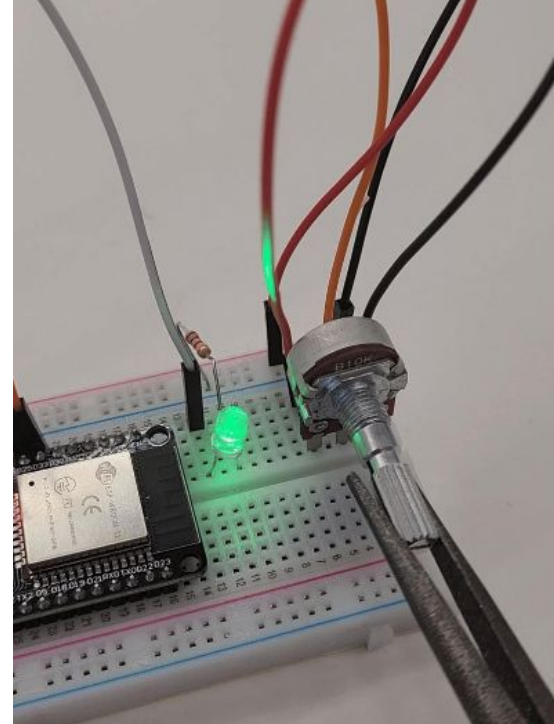
Blink LED



Button LED



Dim LED

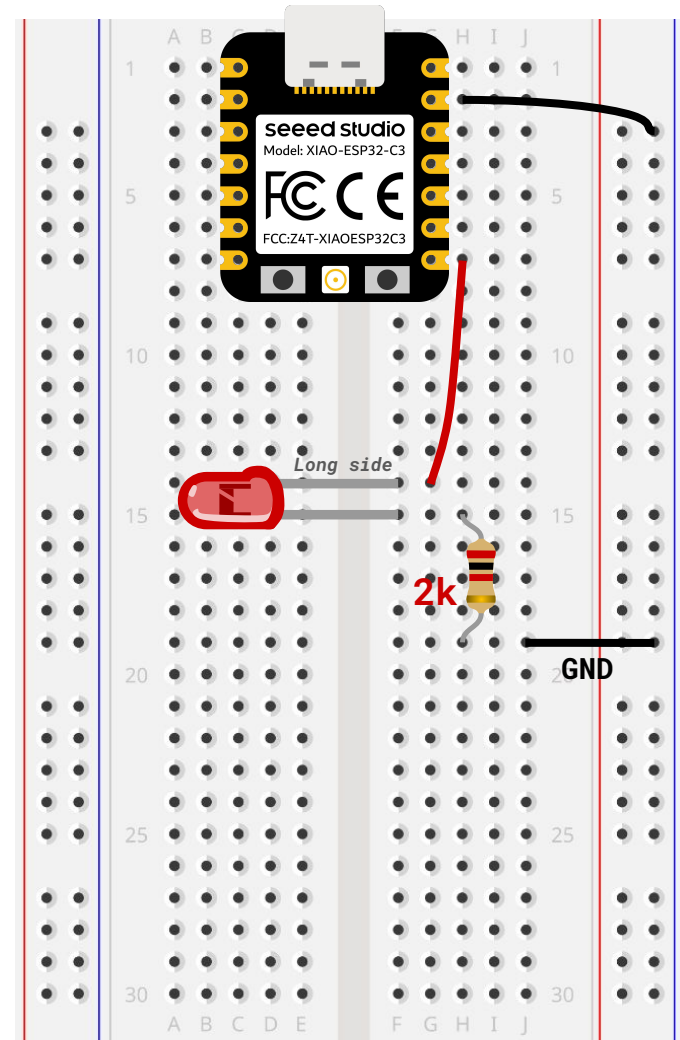




Wiring Diagrams

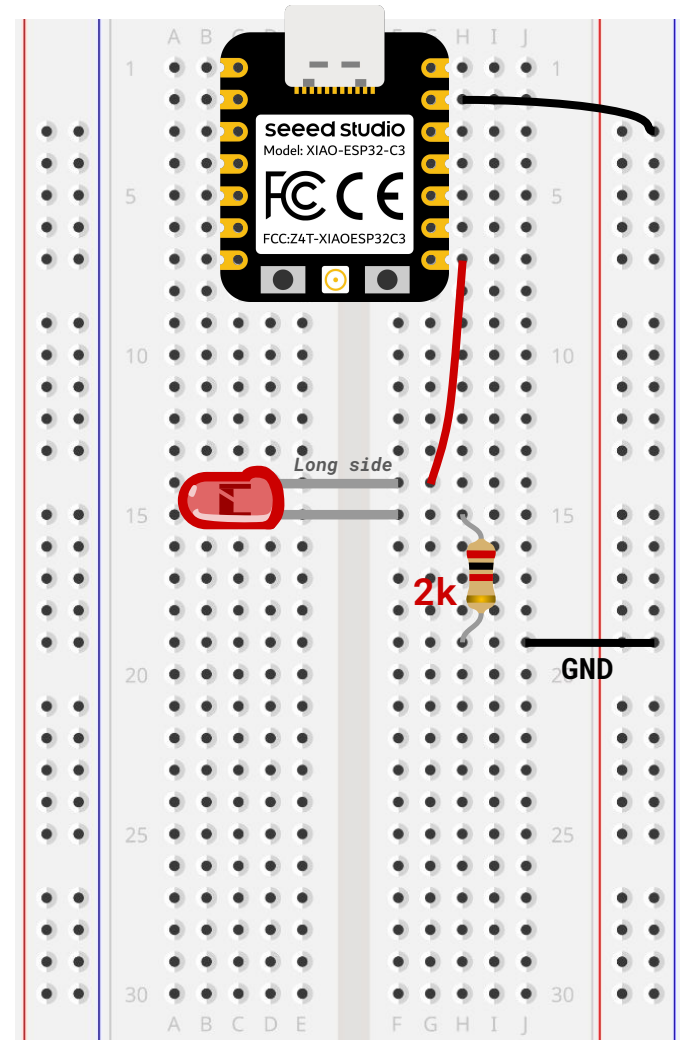
Blinking LED

```
int ledPin = D7;  
int onTime = 500;  
int offTime = 1000;  
  
void setup() {  
    pinMode(ledPin, Which Mode?);  
}  
  
void loop() {  
    // Turn on  
    digitalWrite(ledPin, HIGH or LOW?);  
    delay(onTime);  
    // Turn off  
    digitalWrite(ledPin, HIGH or LOW?);  
    delay(offTime);  
}
```



Blinking LED Solution

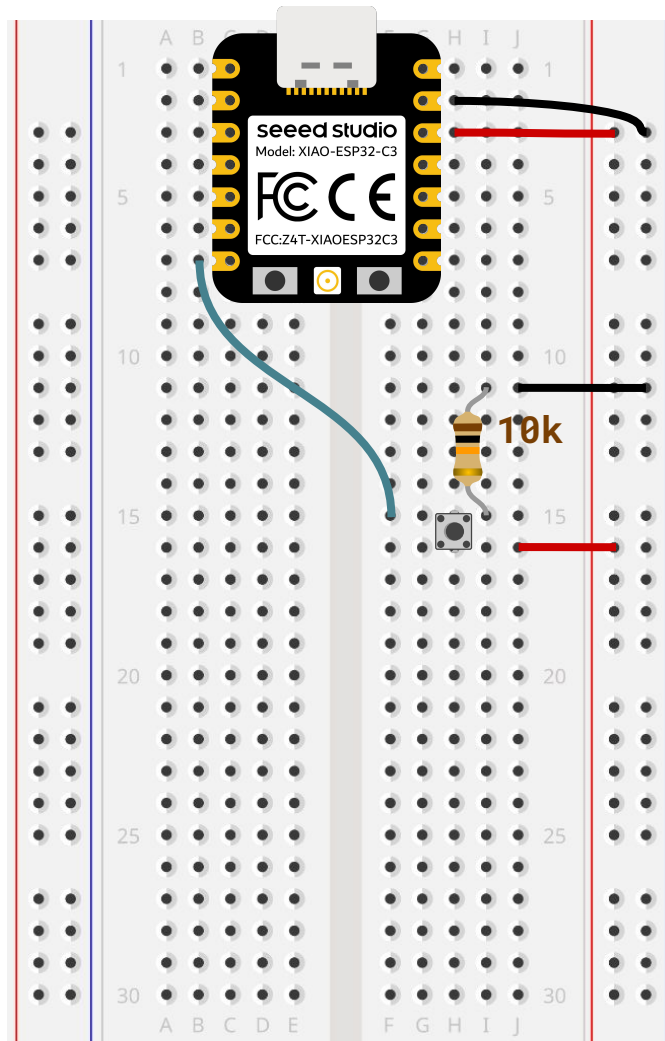
```
int ledPin = D7;  
int onTime = 500;  
int offTime = 1000;  
  
void setup() {  
    pinMode(ledPin, OUTPUT);  
}  
  
void loop() {  
    // Turn on  
    digitalWrite(ledPin, HIGH);  
    delay(onTime);  
    // Turn off  
    digitalWrite(ledPin, LOW);  
    delay(offTime);  
}
```



Pull-Down Resistor

*When the button isn't pressed,
digitalRead reads **LOW** or 0.*

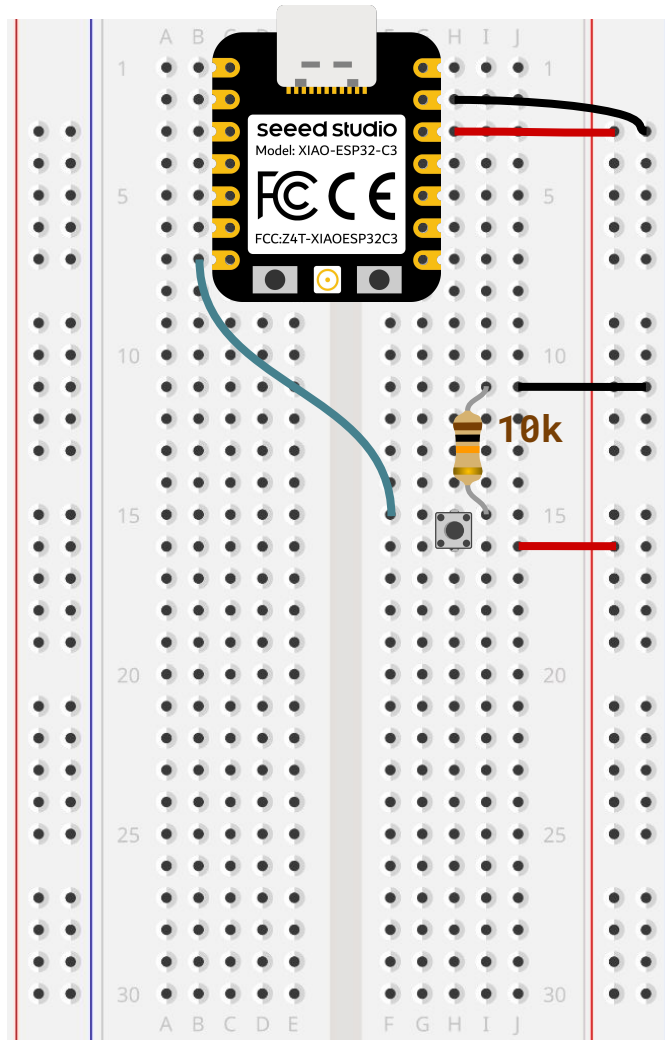
```
int buttonPin = D6;  
int val;  
  
void setup() {  
    Serial.begin(9600);  
    pinMode(???,  
}  
void loop() {  
    val = ???Read(???)  
    Serial.println(???)  
}
```



Pull-Down Resistor Solution

*When the button isn't pressed,
digitalRead reads **LOW** or 0.*

```
int buttonPin = D6;  
int val;  
  
void setup() {  
  Serial.begin(9600);  
  pinMode(buttonPin, INPUT);  
}  
void loop() {  
  val = digitalRead(buttonPin);  
  Serial.println(val);  
}
```

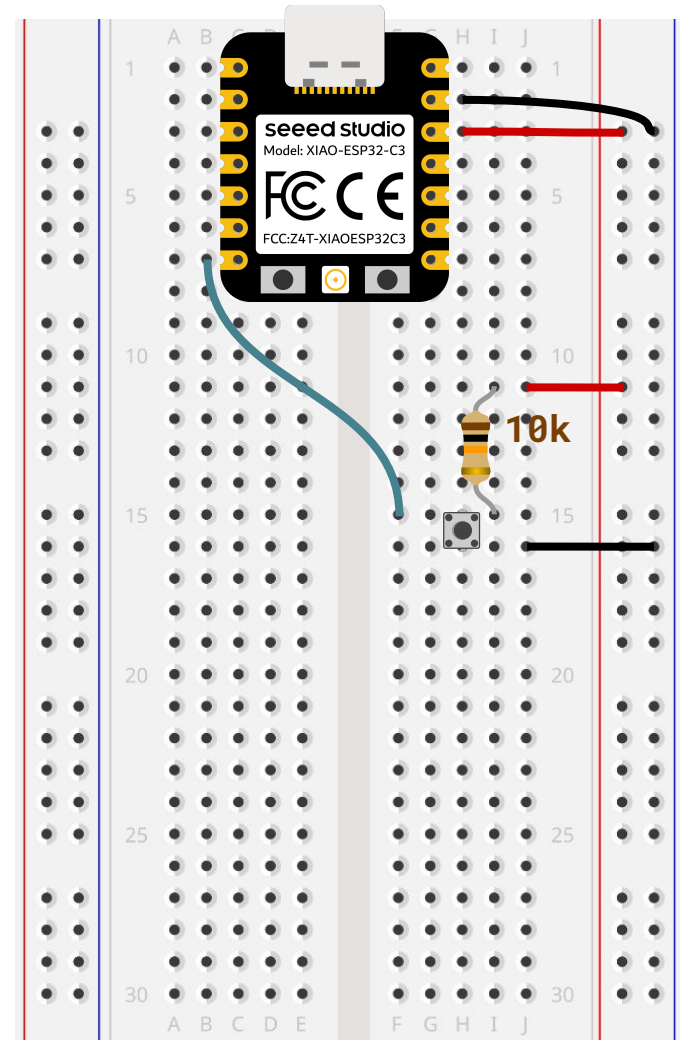


Pull-Up Resistor

When the button isn't pressed, `digitalRead` reads **HIGH** or **1**. What change in code?

```
int buttonPin = D6;  
int val;
```

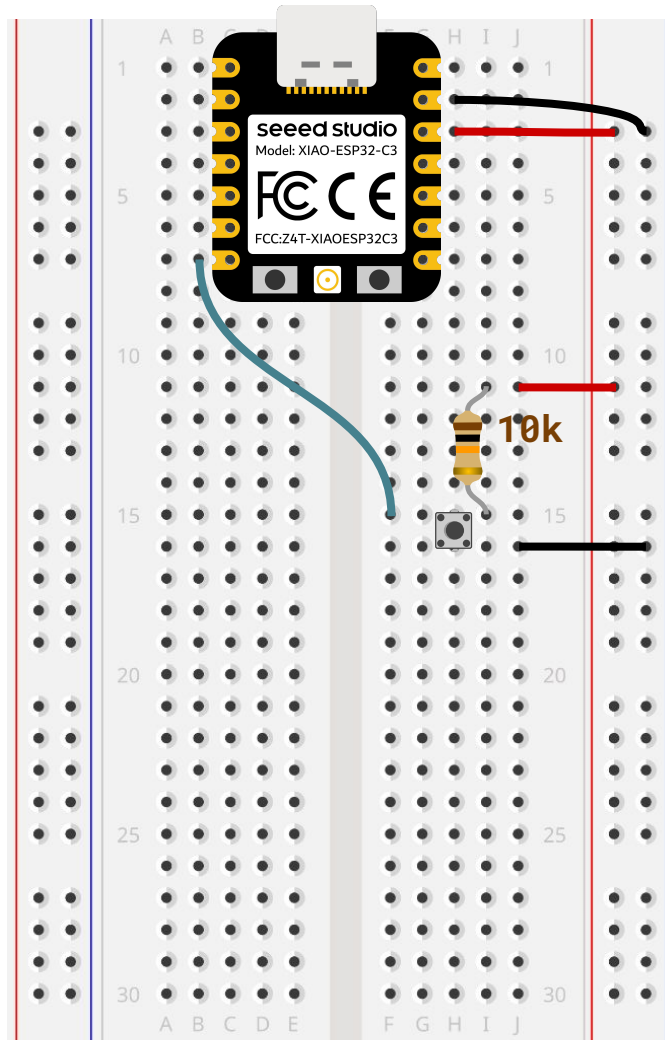
```
void setup() {  
    ???  
}  
void loop() {  
    ???  
}
```



Pull-Up Resistor Solution

*When the button isn't pressed, digitalRead reads **HIGH** or 1. No change in code.*

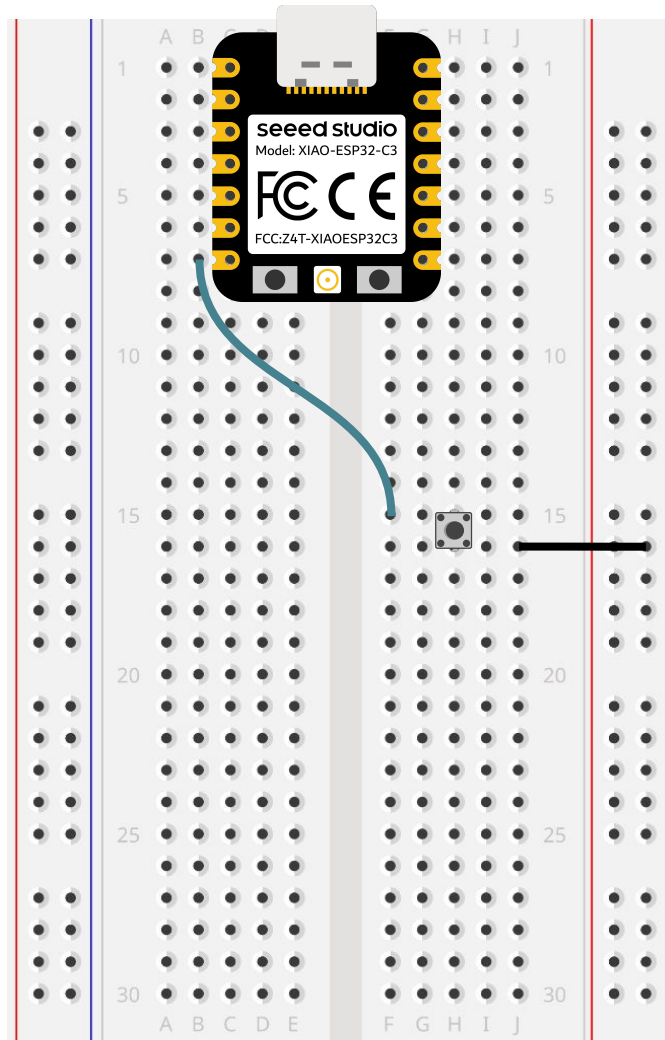
```
int buttonPin = D6;  
int val;  
  
void setup() {  
    Serial.begin(9600);  
    pinMode(buttonPin, INPUT);  
}  
void loop() {  
    val = digitalRead(buttonPin);  
    Serial.println(val);  
}
```



Internal Pull-Up Resistor

Your Xiao has an internal pullup resistor. We can make the button circuit without the physical resistor!

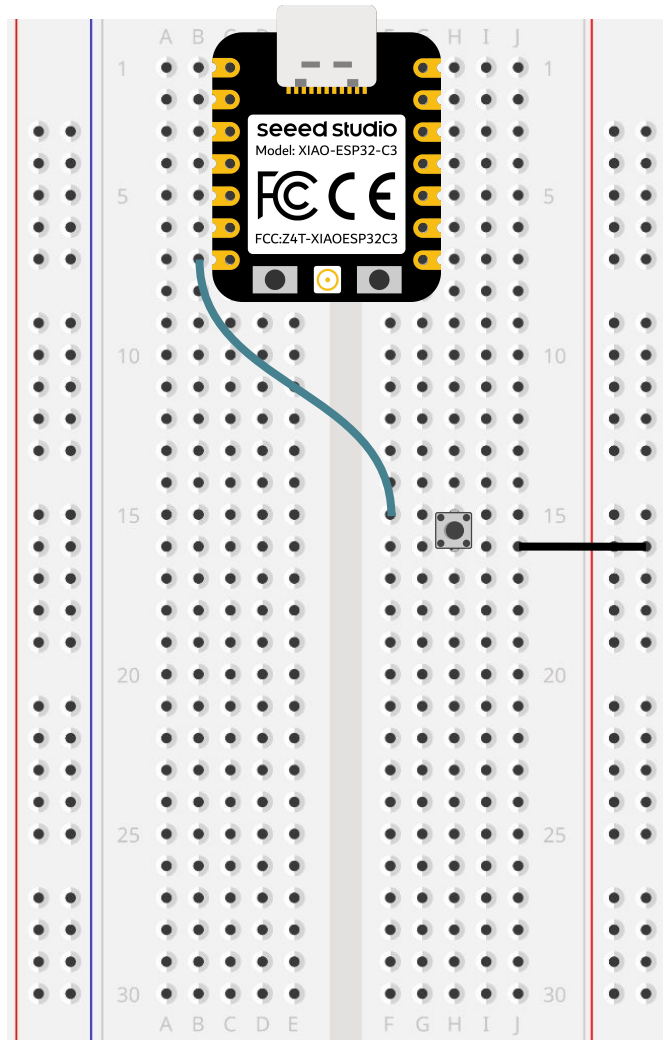
```
int buttonPin = ???;  
int val;  
  
void setup() {  
  Serial.begin(9600);  
  pinMode(???, INPUT_PULLUP);  
}  
  
void loop() {  
  val = // Get value of button  
  // Print value of button  
}
```



Internal Pull-Up Resistor Sol.

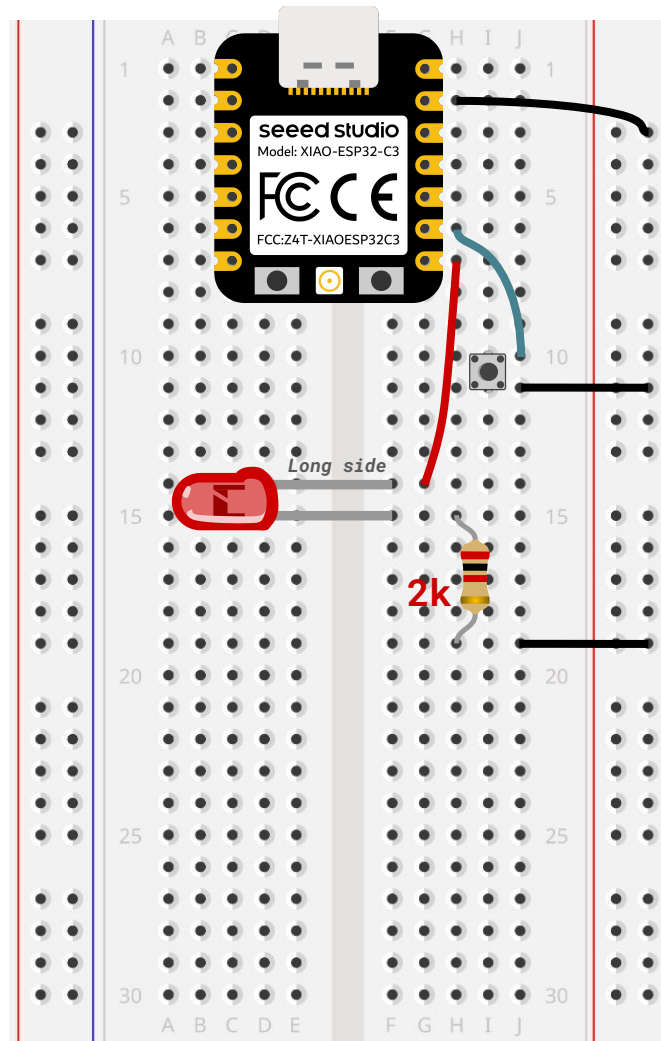
Your Xiao has an internal pullup resistor. We can make the button circuit without the physical resistor!

```
int buttonPin = D6;  
int val;  
  
void setup() {  
  Serial.begin(9600);  
  pinMode(buttonPin, INPUT_PULLUP);  
}  
  
void loop() {  
  val = digitalRead(buttonPin);  
  Serial.println(val);  
}
```



Button Controlled LED

```
int ledPin = ???;  
int buttonPin = ???;  
int val;  
  
void setup() {  
  pinMode(???, OUTPUT);  
  pinMode(???, INPUT_PULLUP);  
}  
  
void loop() {  
  val = ???;  
  
  if (???) {  
    ???(ledPin, HIGH);  
  } else {  
    ???(ledPin, LOW);  
  }  
}
```



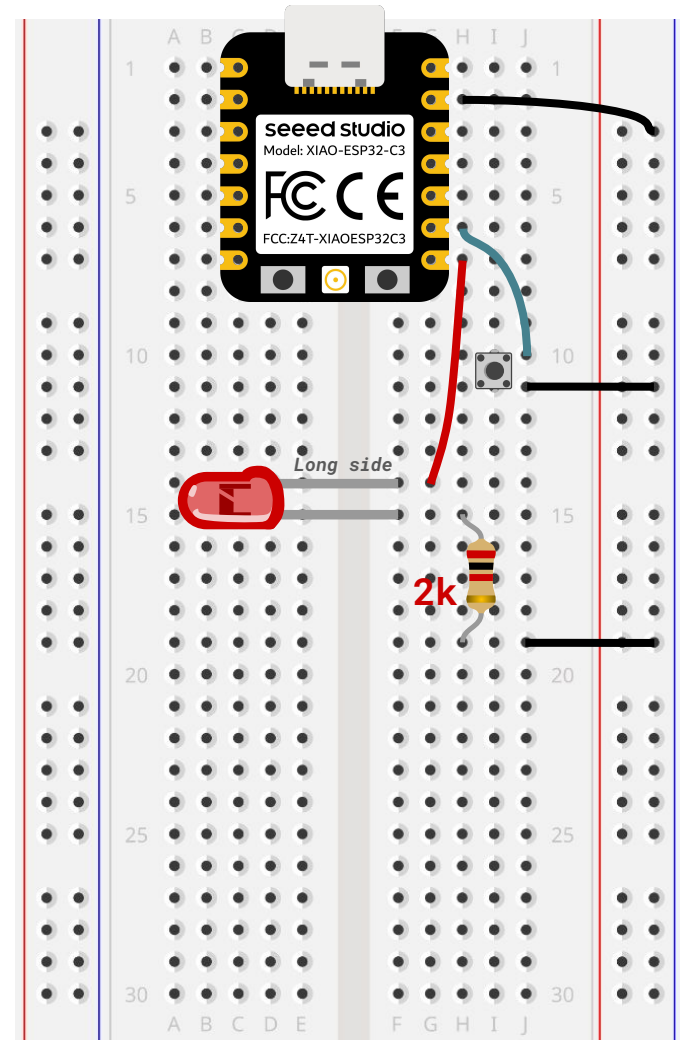
Button Controlled LED Sol.

```
int ledPin = D7;
int buttonPin = D8;
int val;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
}

void loop() {
  val = digitalRead(buttonPin);

  if (val == LOW) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
}
```

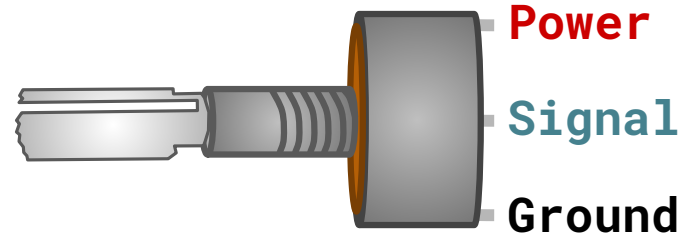


About Potentiometers

A variable resistor. By turning the knob, you vary the resistance which in turn varies the amount of voltage allowed through.

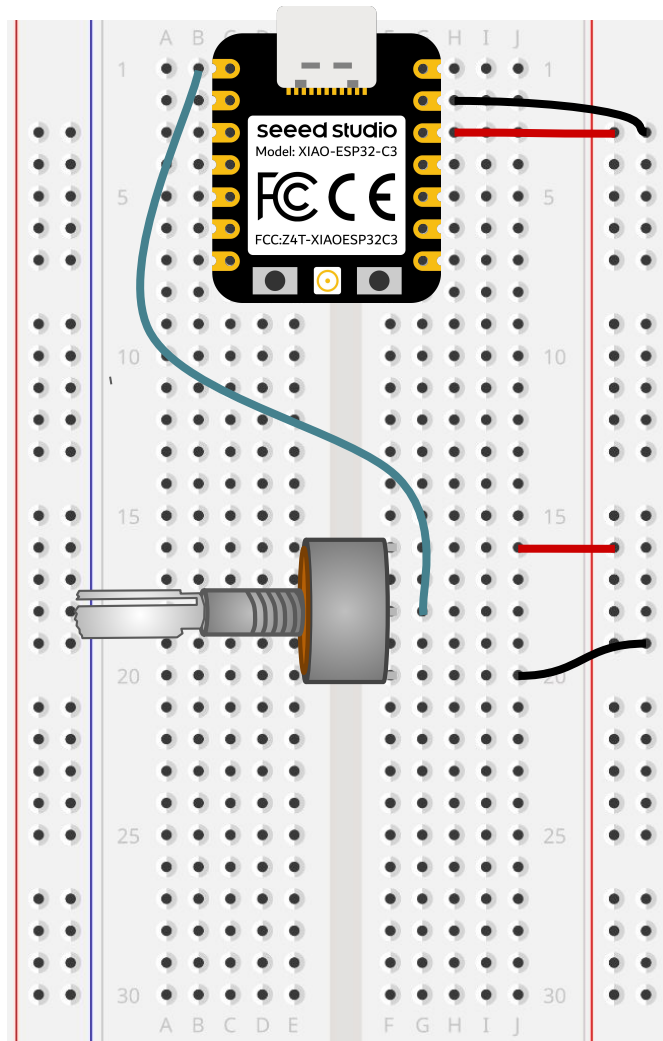
Can be useful for speed control, dimming lights.

Variable signal... so analog!



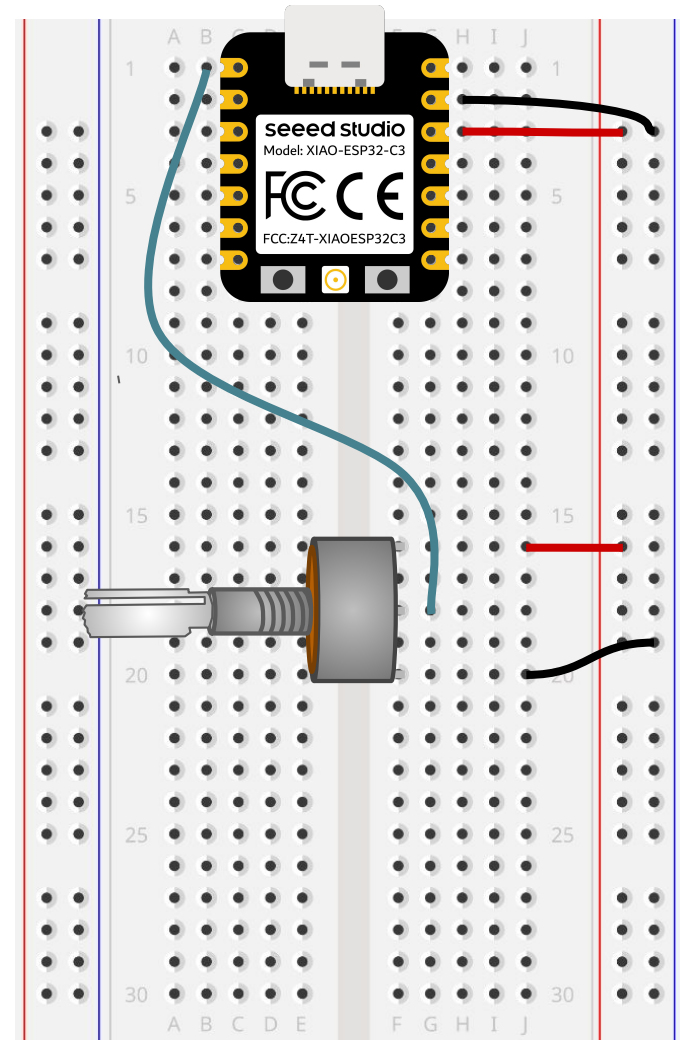
Potentiometer

```
int potPin = ???;  
int potVal;  
  
void setup() {  
  Serial.begin(9600);  
  pinMode(potPin, ???);  
}  
  
void loop() {  
  potVal = ???(potPin);  
  // Print potVal  
}
```



Potentiometer Solution

```
int potPin = A0;  
int potVal;  
  
void setup() {  
  Serial.begin(9600);  
  pinMode(potPin, INPUT);  
}  
  
void loop() {  
  potVal = analogRead(potPin);  
  Serial.println(potVal);  
}
```



Mapping Values

Arduino has an `analogRead` range from 0 to 4095, and an `analogWrite` range only from 0 to 255.

Potentiometer has a range of 0 to 4095.

LED has range of 0 to 255.

Thus, we use the `map()` function.

```
map(value, fromLow, fromHigh, toLow, toHigh)
```

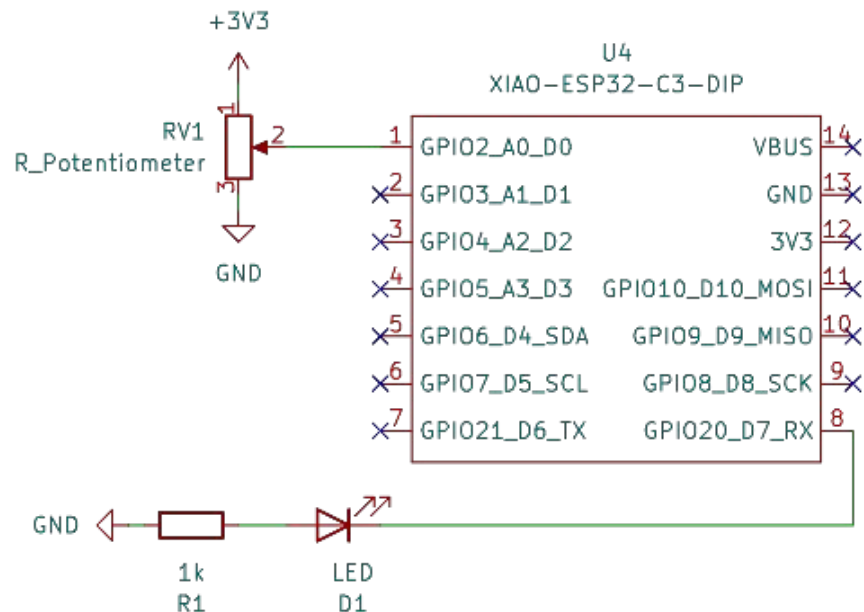
Dimmable LED

```
int potPin = A0;
int ledPin = D7;
int potVal;
int ledVal;

const int freq = 5000;
const int resolution = 8;

void setup() {
  // Set potPin to input or output?
  // Set ledPin to input or output?
  ledcAttach(ledPin, freq, resolution);
}

void loop() {
  potVal = // Get value of potentiometer
  ledVal = map(???);
  ledcWrite(ledPin, ledVal);
}
```



Dimmable LED Solution

```
int potPin = A0;  
int ledPin = D7;  
int potVal;  
int ledVal;  
  
const int freq = 5000;  
const int resolution = 8;  
  
void setup() {  
  pinMode(potPin, INPUT);  
  pinMode(ledPin, OUTPUT);  
  ledcAttach(ledPin, freq, resolution);  
}  
void loop() {  
  potVal = analogRead(potPin);  
  ledVal = map(potVal, 0, 4095, 0, 255);  
  ledcWrite(ledPin, ledVal);  
}
```

